check_paired.py 코드 해설

> lab11 · 생물정보학실험
> 대상 파일: `lab11/src/check_paired.py`
> 실행 방법과 결과 해석은 `check_paired_usage.md`를 참고하세요.

---

1. 코드 전문

```
import sys

def normalize_name(header):
    name = header.strip().split()[0]
    name = name.lstrip("@")

    # 끝의 /1, /2 또는 _1, _2 제거
    if name.endswith(("/1", "/2", "_1", "_2")):
        name = name[:-2]

    return name


def check_paired(file1, file2):
    count = 0
    mismatch = 0

    with open(file1) as f1, open(file2) as f2:
        while True:
            h1 = f1.readline()
            h2 = f2.readline()

            if not h1 and not h2:
                break

            if not h1 or not h2:
                print("두 파일의 read 개수가 다릅니다.")
                return

            # 각 read의 나머지 3줄 건너뛰기
            for _ in range(3):
                if not f1.readline() or not f2.readline():
                    print("불완전한 FASTQ 레코드가 있습니다.")
                    return

            count += 1
```

```
            name1 = normalize_name(h1)
            name2 = normalize_name(h2)

            if name1 != name2:
                mismatch += 1
                if mismatch <= 5:
                    print("불일치:", name1, name2)

    print("확인한 read pair 수:", count)
    print("이름 불일치 수:", mismatch)

    if mismatch == 0:
        print("모든 read가 같은 순서로 짝을 이룹니다.")
    else:
        print("일부 read의 짝이 일치하지 않습니다.")


check_paired(sys.argv[1], sys.argv[2])
```

코드는 크게 다섯 부분으로 나뉩니다.

부분	하는 일
① 준비	`sys` 모듈 불러오기
② `normalize_name()` 함수	헤더 줄에서 비교용 read 이름만 뽑아내기
③ `check_paired()` — 두 파일 함께 읽기	R1, R2를 4줄씩 동시에 읽으며 개수와 레코드 형식 확인
④ `check_paired()` — 이름 비교와 출력	같은 위치의 read 이름 비교, 결과 출력
⑤ 실행	명령줄의 두 파일 이름으로 `check_paired()` 호출

이 프로그램이 확인하는 두 가지

paired-end 데이터가 올바르려면 아래 두 조건을 **모두** 만족해야 합니다. 이 프로그램은 이 두 가지를 차례로 확인합니다.

검사 1. 두 파일의 read 개수가 같은가?

R1의 n번째 read와 R2의 n번째 read가 한 쌍이므로, 두 파일의 read 수가 다르면 짝이 없는 read가 생깁니다.

- 두 파일에서 4줄(레코드 1개)씩 **동시에** 읽어 나갑니다.
- 두 파일이 **같은 시점에** 끝나면 → 개수가 같음 (`count`가 곧 read 개수)
- **한쪽 파일만 먼저** 끝나면 → `"두 파일의 read 개수가 다릅니다."` 출력 후 종료

```
if not h1 and not h2:      # 둘 다 끝남 → 개수 같음
    break
if not h1 or not h2:       # 한쪽만 끝남 → 개수 다름
```

```
    print("두 파일의 read 개수가 다릅니다.")
    return
```

검사 2. 같은 순서의 read끼리 이름이 실제로 짝을 이루는가?

개수가 같아도 순서가 뒤섞였을 수 있으므로, 같은 위치에 있는 R1·R2 read의 이름을 하나씩 비교합니다. 이때 R1/R2를 구분하려고 이름 **끝에 붙인 숫자나 기호**(`/1`, `/2`, `_1`, `_2`)는 **빼고** 비교합니다.

```
R1 이름        R2 이름        끝 두 글자 제외 후      판정
read123/1      read123/2      read123 = read123     짝 맞음 ✔
read123_1      read123_2      read123 = read123     짝 맞음 ✔
read123/1      read124/2      read123 ≠ read124     불일치 ✘
```

```
name1 = normalize_name(h1)   # 끝의 /1, _1 등을 뗀 이름
name2 = normalize_name(h2)   # 끝의 /2, _2 등을 뗀 이름
if name1 != name2:           # 남은 이름이 다르면 짝이 어긋난 것
    mismatch += 1
```

- 끝 표시를 떼는 일은 `normalize_name()`이 맡습니다. (→ 3장)
- 모든 쌍을 다 비교한 뒤 불일치가 0개면 `"모든 read가 같은 순서로 짝을 이룹니다."`를 출력합니다.

검사	확인 내용	담당 코드	문제가 있을 때 출력
검사 1	R1, R2의 read 개수가 같은가	`while` 반복 안의 `if not h1 ...`	두 파일의 read 개수가 다릅니다.
검사 2	같은 순서의 read 이름이 (끝 숫자·기호 제외하고) 같은가	`normalize_name()` + `if name1 != name2`	불일치: … / 일부 read의 짝이 일치하지 않습니다.

---

2. ① 준비

```
import sys
```

코드	해설
(맨 윗줄 빈 줄)	파일 첫 줄이 비어 있지만 실행에는 아무 영향이 없다.
`import sys`	명령줄에서 입력한 값(파일 이름 두 개)을 읽기 위해 `sys` 모듈을 불러온다. 나중에 `sys.argv[1]`, `sys.argv[2]`로 꺼낸다.

---

3. ② `normalize_name()` 함수

```
def normalize_name(header):
    name = header.strip().split()[0]
    name = name.lstrip("@")

    # 끝의 /1, /2 또는 _1, _2 제거
    if name.endswith(("/1", "/2", "_1", "_2")):
        name = name[:-2]

    return name
```

코드	해설
`def normalize_name(header):`	FASTQ 헤더 줄(1번째 줄) 하나를 받아 비교용 이름을 돌려주는 함수.
`header.strip()`	앞뒤 공백과 줄바꿈(`\n`)을 지운다.
`.split()`	공백을 기준으로 잘라 리스트로 만든다.
`[0]`	그중 **첫 번째 단어**만 가져온다. 뒤에 붙은 `1 length=101` 같은 부가 정보는 버린다.
`name.lstrip("@")`	맨 앞의 `@`를 지운다. (`lstrip`은 왼쪽 끝에서 해당 문자를 **모두** 지운다.)
`name.endswith(("/1", "/2", "_1", "_2"))`	`endswith()`에 **튜플**을 주면 그중 **하나라도** 끝에 있으면 `True`가 된다.
`name = name[:-2]`	슬라이싱으로 **마지막 두 글자**를 잘라 낸다. `/1`, `_2` 등은 모두 두 글자이므로 정확히 꼬리표만 지워진다.
`return name`	정리된 이름을 돌려준다.

변환 예시:

```
입력                              strip().split()[0]   lstrip("@")       [:-2]
"@SRR1553607.1 1 length=101\n"  → "@SRR1553607.1"    → "SRR1553607.1"   (해당 없음)
"@read123/1\n"                  → "@read123/1"       → "read123/1"      → "read123"
"@read123_2 extra\n"            → "@read123_2"       → "read123_2"      → "read123"
```

> 꼬리표를 지우는 이유: R1은 `read123/1`, R2는 `read123/2`처럼 끝만 다르게 붙이는 경우가 많습니다. 그대로 비교하면 같은 쌍인데도 "다르다"고 판단하므로, 공통 부분(`read123`)만 남겨 비교합니다.

---

4. ③ `check_paired()` — 두 파일 함께 읽기

```
def check_paired(file1, file2):
    count = 0
    mismatch = 0

    with open(file1) as f1, open(file2) as f2:
        while True:
            h1 = f1.readline()
            h2 = f2.readline()
```

코드	해설
`def check_paired(file1, file2):`	R1, R2 파일 이름 두 개를 받는 함수.
`count = 0`	검사한 read **쌍(pair)** 의 수를 셀 변수.
`mismatch = 0`	이름이 서로 다른 쌍의 수를 셀 변수.
`with open(file1) as f1, open(file2) as f2:`	두 파일을 **한 번에** 연다. 블록이 끝나면 둘 다 자동으로 닫힌다.
`while True:`	끝이 정해지지 않은 반복. 안에서 `break` 나 `return` 을 만날 때까지 계속 돈다.
`h1 = f1.readline()`	R1에서 **한 줄만** 읽는다. 지금 위치는 항상 레코드의 1번째 줄(헤더)이다.
`h2 = f2.readline()`	R2에서도 같은 위치의 헤더 한 줄을 읽는다.

> `readlines()` (모든 줄을 한꺼번에)가 아니라 `readline()` (한 줄씩)을 쓰기 때문에, 파일이 아무리 커도 메모리를 거의 쓰지 않습니다. `gc_content.py` 등의 `read_fastq()` 와 다른 점입니다.

```
            if not h1 and not h2:
                break

            if not h1 or not h2:
                print("두 파일의 read 개수가 다릅니다.")
                return
```

코드	해설
`readline()` 의 반환값	파일 끝에 도달하면 빈 문자열 `""` 을 돌려준다. 빈 문자열은 `False` 로 취급되므로 `not h1` 은 "R1이 끝났다"는 뜻이다.
`if not h1 and not h2:`	**두 파일이 동시에** 끝났으면 개수가 같다는 뜻 → `break` 로 반복을 빠져나가 결과 출력으로 간다.
`if not h1 or not h2:`	**한쪽만** 끝났으면 개수가 다르다는 뜻이다.

코드	해설
`return`	메시지를 출력하고 함수를 바로 끝낸다. 그래서 이 경우에는 pair 수, 불일치 수가 **출력되지 않는다.**

두 `if`의 순서가 중요합니다. “둘 다 끝남”을 먼저 검사하지 않으면, 정상적으로 끝난 경우도 “한쪽이 끝남”에 걸려 잘못된 경고가 나옵니다.

```
        # 각 read의 나머지 3줄 건너뛰기
        for _ in range(3):
            if not f1.readline() or not f2.readline():
                print("불완전한 FASTQ 레코드가 있습니다.")
                return

        count += 1
```

코드	해설
`for _ in range(3):`	3번 반복한다. 반복 변수를 쓰지 않으므로 이름을 `_`로 둔다.
`f1.readline()`, `f2.readline()`	서열, `+`, 품질 줄을 읽기만 하고 **저장하지 않는다** (= 건너뛰기). 이 프로그램은 이름만 비교하면 되기 때문이다.
`if not ... or not ...:`	어느 한쪽이라도 줄이 모자라면 레코드가 4줄을 다 못 채운 것이다.
`print(...)`, `return`	“불완전한 FASTQ 레코드” 경고를 출력하고 끝낸다.
`count += 1`	4줄을 무사히 다 읽었으므로 쌍 하나를 센다.

FASTQ 4줄 중 실제로 쓰는 줄:

```
@SRR1553607.1 1 length=101   ← h1 / h2 로 읽어 비교에 사용
GTTAGCGTTGTTGATCGC...        ┐
+SRR1553607.1 1 length=101   ├ for 반복으로 읽고 버림
BB@FFFFFHHHHHJJJJJ...        ┘
```

---

5. ④ `check_paired()` — 이름 비교와 출력

```
        name1 = normalize_name(h1)
        name2 = normalize_name(h2)

        if name1 != name2:
            mismatch += 1
```

```
                if mismatch <= 5:
                    print("불일치:", name1, name2)
```

코드	해설
`name1 = normalize_name(h1)`	R1 헤더에서 비교용 이름을 뽑는다.
`name2 = normalize_name(h2)`	R2 헤더에서도 똑같이 뽑는다.
`if name1 != name2:`	같은 순서의 두 read 이름이 **다르면** 짝이 어긋난 것이다.
`mismatch += 1`	불일치 수를 1 늘린다. 몇 개든 **끝까지 모두 센다.**
`if mismatch <= 5:`	불일치가 수천 개일 수 있으므로, 화면에는 **처음 5개만** 보여 준다.
`print("불일치:", name1, name2)`	어떤 이름끼리 어긋났는지 출력한다.

> 불일치가 나와도 반복은 멈추지 않습니다. 끝까지 읽어서 전체 불일치 수를 알려 줍니다.

```
    print("확인한 read pair 수:", count)
    print("이름 불일치 수:", mismatch)

    if mismatch == 0:
        print("모든 read가 같은 순서로 짝을 이룹니다.")
    else:
        print("일부 read의 짝이 일치하지 않습니다.")
```

코드	해설
들여쓰기 위치	`while` 과 `with` 블록 **밖**에 있으므로, `break` 로 반복이 정상 종료된 뒤에만 실행된다.
`print("확인한 read pair 수:", count)`	검사한 쌍의 수. 곧 R1(=R2)의 read 개수다.
`print("이름 불일치 수:", mismatch)`	이름이 다른 쌍의 수.
`if mismatch == 0:`	불일치가 하나도 없으면 정상 판정을 출력한다.
`else:`	하나라도 있으면 “일부 read의 짝이 일치하지 않습니다.”를 출력한다.

---

6. ⑤ 실행

```
check_paired(sys.argv[1], sys.argv[2])
```

코드	해설
`sys.argv[1]`	명령줄의 첫 번째 값 → R1 파일.

코드	해설
`sys.argv[2]`	명령줄의 두 번째 값 → R2 파일.
`check_paired(...)`	두 파일 이름으로 검사 함수를 실행한다. 파일을 하나라도 빼먹으면 `IndexError`가 난다.

```
python3 src/check_paired.py data/SRR1553607_1.fastq data/SRR1553607_2.fastq
#          sys.argv[0]          sys.argv[1]             sys.argv[2]
```

출력 예시 (실습 데이터):

```
확인한 read pair 수: 203445
이름 불일치 수: 0
모든 read가 같은 순서로 짝을 이룹니다.
```

---

7. 한눈에 보는 흐름

```
R1 파일 ──┐
          ├── 헤더 한 줄씩 동시에 읽기 (h1, h2)
R2 파일 ──┘
    │
    ├── 둘 다 끝? ──────────────────────────→ 반복 종료 → 결과 출력
    ├── 한쪽만 끝? ─────────────────────────→ "read 개수가 다릅니다." 후 종료
    │
    ├── 나머지 3줄 건너뛰기
    │     줄이 모자람? ──────────────→ "불완전한 FASTQ 레코드" 후 종료
    │
    ├── count += 1
    ├── normalize_name(h1) vs normalize_name(h2)
    │     다르면 mismatch += 1 (처음 5개는 화면에 출력)
    │
    └── 다음 레코드로 (반복)

결과 출력: pair 수, 불일치 수, 최종 판정
```

---

8. 알아 두면 좋은 점

- **메모리를 적게 씁니다.** 한 줄씩 읽고 바로 버리기 때문에 수백만 read짜리 파일도 문제없이 검사할 수 있습니다.
- **헤더만 비교합니다.** 서열이나 품질이 올바른지는 검사하지 않습니다. 또 헤더 줄이 실제로 `@`로 시작하는지도 확인하지 않습니다.
- **`_1`, `_2` 제거는 단순한 규칙입니다.** 이름이 원래 `sample_1`처럼 끝나는 경우에도 꼬리표로 보고 지웁니다. 다만 R1과 R2에 같은 규칙이 적용되므로 비교 결과에는 보통 문제가 없습니다.

- **개수가 다르면 통계가 나오지 않습니다.** 그 경우 `return`으로 바로 끝나므로 그때까지 센 pair 수나 불일치 수는 출력되지 않습니다.
- **함수 호출이 파일 맨 아래에 그대로 있습니다.** `if __name__ == "__main__":`으로 감싸지 않았기 때문에, 다른 파일에서 `import check_paired`를 하면 바로 실행되어 버립니다.
- **압축 파일(`.fastq.gz`)은 읽지 못합니다.** 압축을 푼 파일을 넣어야 합니다.